Subject: PFP191 – Programming Fundamentals with Python
Topic: Student Management System

Objectives

Design and implement a Python application to manage student records and course enrollment at an educational institution, applying knowledge gained from the course, including:

- Object-Oriented Programming (OOP).
- Data management using lists or dictionaries.
- File processing (read/write text files).
- Exception Handling.
- Modular Programming (multiple .py files).

Group Work Guidelines

- **Group size:** 3–4 students.
- **Duration:** 4–6 weeks.
- **Deliverables:**
  - Source code (.py).
  - Technical report (REPORT.pdf).
  - Presentation slides (SLIDES.pptx).
- **Submission:** Via LMS as directed by the instructor.

---

1. **Student Management**

   Each student contains:

   - student_id
   - full_name
   - date_of_birth
   - gender
   - class_name
   - phone
   - email
   - status

   Example

   - student_id: SE00001, SE00002, SE00003, SE00004...
   - full_name: Nguyen Van An, Tran Thi Thao, Le Minh Chau...
   - date_of_birth: 15/03/2005
   - gender: Male / Female
   - class_name: SE1901, IT1902, AI1903...
   - phone: 0901234567
   - email: an.nguyen@fpt.edu.vn
   - status: Active / Inactive / Graduated
   - *Note: Two different students must have different student_ids.*

Functional Requirements

- Add a new student to the list of students.
- Update student information based on student_id.
- Filter students by class or status.
- Display active students.

---

2. **Result Management**

Each Result contains:

- student_id
- course_id
- course_name
- semester
- credits
- grade_FE
- grade_RE
- status

Example

- student_id: S001
- course_id: PFP191, MAD101, DBI202...
- course_name: Programming Fundamentals with Python
- semester: Fall2026
- credits: 3
- grade_FE: 0–10 (or blank if not yet graded)
- grade_RE: 0–10 (or blank if not yet graded)
- status: Pass/ Fail

- ***Note: Each student is allowed a maximum of two tests per subject.***

Functional Requirements

- Add a student's course grade to the results list.
- Update a student's grade or status for a course based on the student_id and course_id entered by the user.
- Search enrollments by student ID.
- Display exam results for each student. Sort the list by status in descending order.
- Display the list of students with a "Pass" result. A "Pass" result is achieved when 'grade_FE >= 5' or ('grade_FE < 5' and 'grade_RE >= 5').

---

3. File Handling

- Save data student and result to file.
- Load data from file.

4. **Console-Based User Interface**

   Provide a simple, text-based (CLI) menu system for user interaction.

5. **File I/O**

   Save/load the student list and result list data from .txt files.

6. **Object-Oriented Programming**
   - Use classes such as Student and StudentManager.
   - Apply OOP principles: constructors, encapsulation, and inheritance (if needed).
7. **Advanced Features**
   - Organize code into packages: models/, services/, utils/
   - Implement __str__, and use @property for clean object interfaces
   - Exception handling for invalid input and file errors.
   - Use built-in functions for data manipulation.
8. **Technical Requirements**
   - Organize the code into multiple .py files (modular design).
   - Prepare a presentation explaining the architecture and design decisions.

-- End --